

SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de



VERSIONSVERWALTUNG MIT GIT

Themen in dieser Vorlesung



Lernziele

- Versionsmanagement als Werkzeug für die Software-Entwicklung kennen lernen
- Git in einem verteilten Team einsetzen können



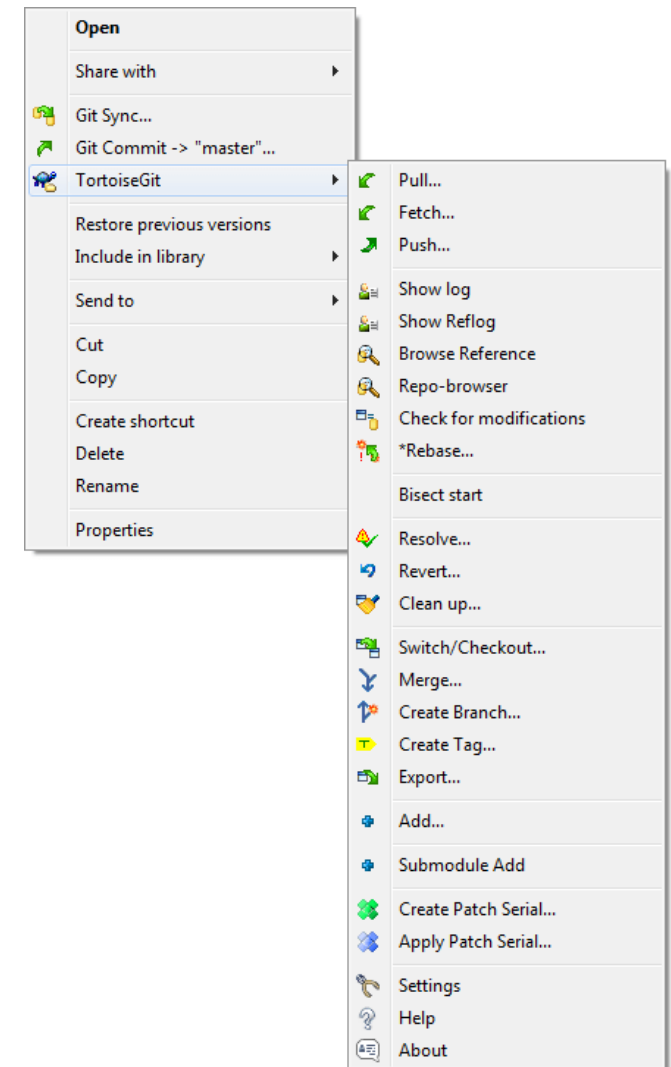
Versionsmanagement mit Git

- Entwickelt 2005 von Linus Torvalds (Linux)
- Ziel: Einsatz bei Linux-Kernel-Entwicklung
- Allgemeine Eigenschaften:
 - Geschwindigkeit
 - Einfaches Design
 - Gute Unterstützung von nicht-linearer Entwicklung (tausende paralleler *Branches*, d.h. verschiedener Verzweigungen der Versionen)
- Vollständig verteilt

<http://git-scm.com/book/de/Los-geht%27s-Die-Geschichte-von-Git>

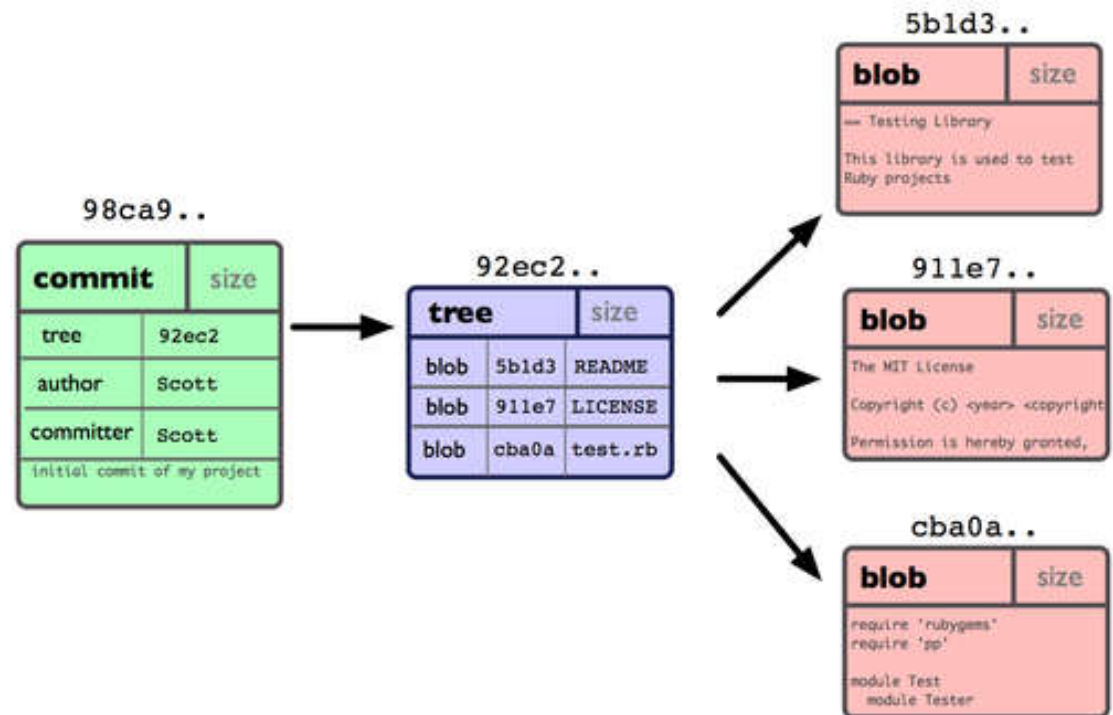
Git - Links

- Git-Website <http://git-scm.com/>
- Downloads
 - <http://git-scm.com/download/win>
- Integration in den Windows Explorer
 - <https://download.tortoisegit.org/tgit/>
- Integration in Eclipse
 - <http://www.eclipse.org/egit/>

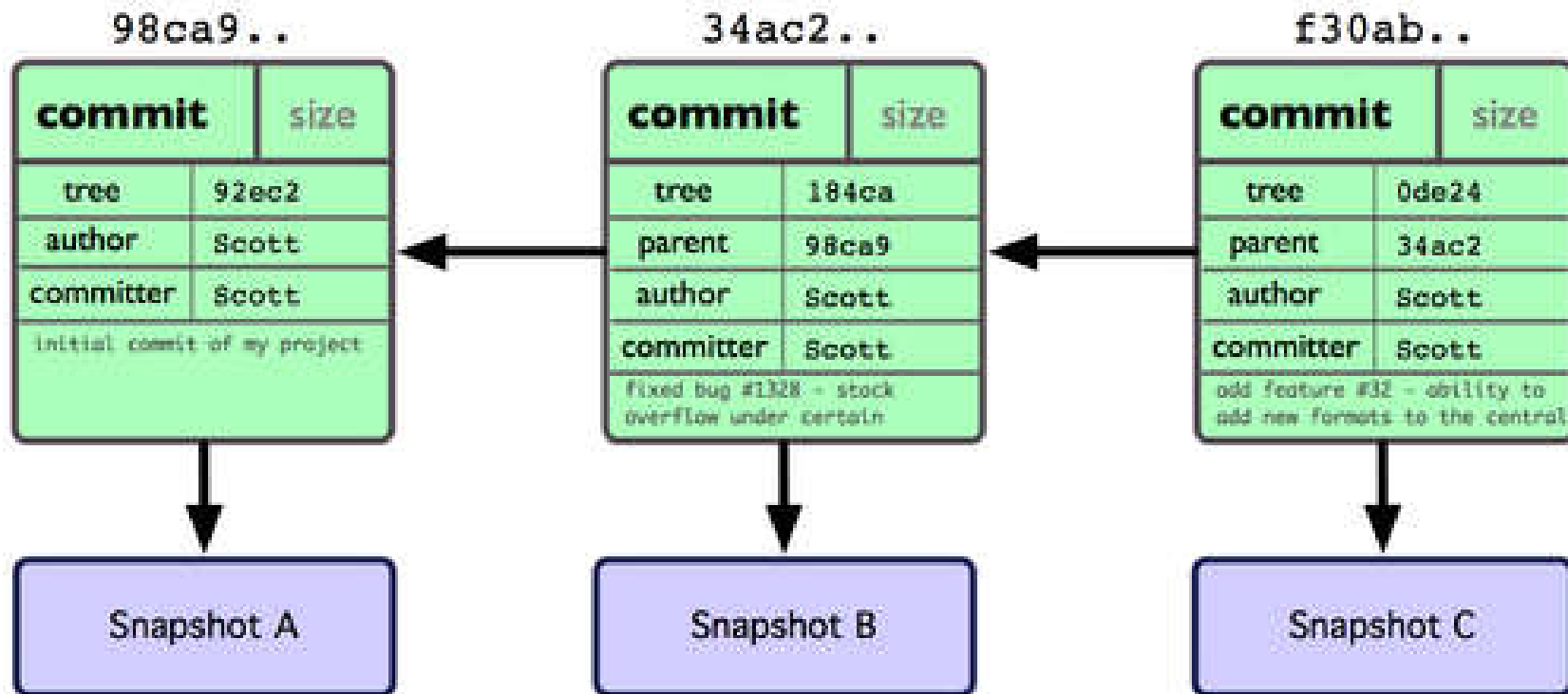


Commits in Git

- Ein Commit ist ein Schnappschuss (Snapshot) von Dateien, baumartig gespeichert
- Neben den Dateiinhalten sind auch Meta-Informationen (Autor, Datum, ...) dort abgelegt



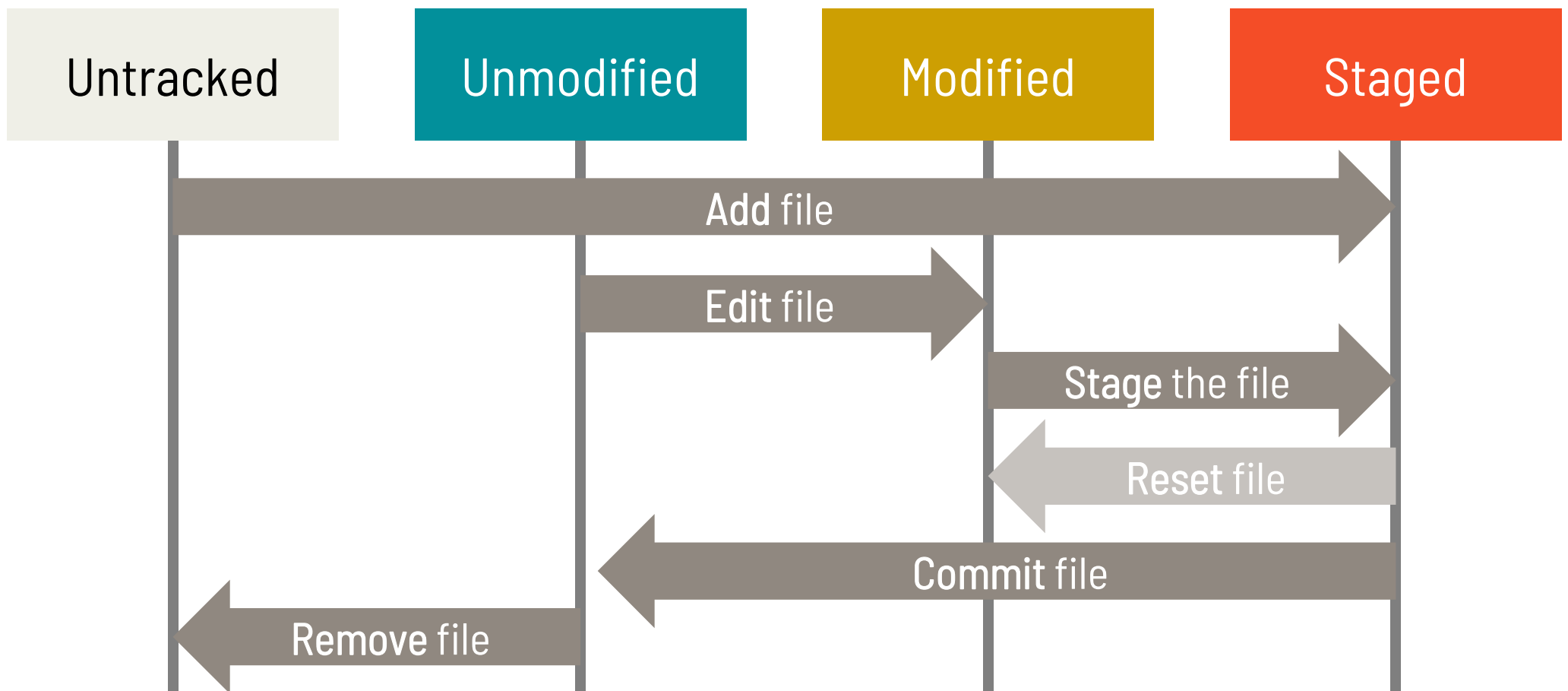
Ablage mehrerer Snapshots in Git



Checksummen bei Git

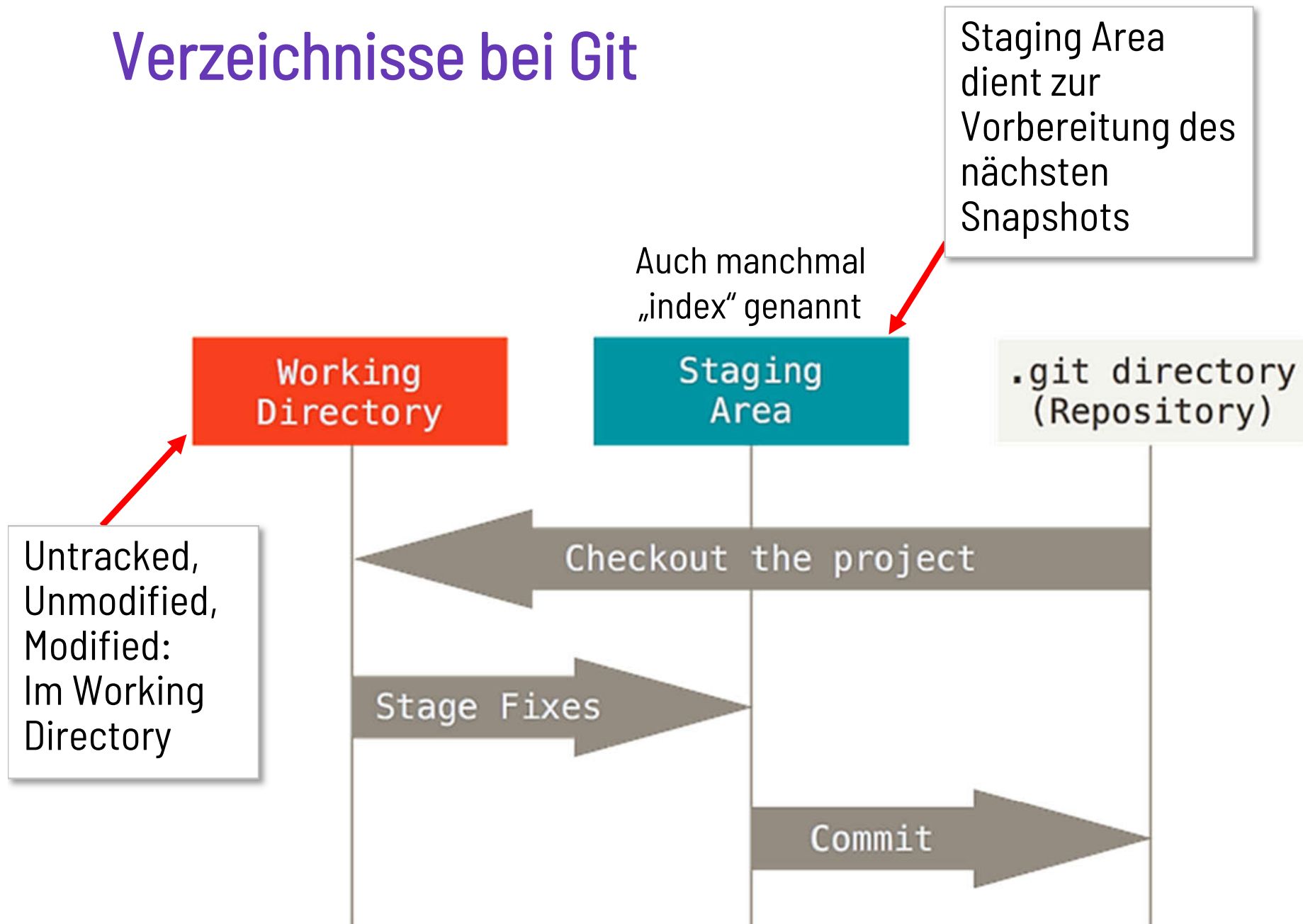
- Alle gespeicherten Daten werden von Git mit einer **Prüfsumme** versehen und hinterher darüber referenziert
 - Damit: Git bemerkt jede Änderung an Dateien oder Verzeichnisse, auch solche durch Hard- oder Softwarefehler oder Übertragungsfehler
- Die Prüfsumme ist ein **SHA-1** Hashwert: Ein String mit 40 Zeichen aus Hexadezimalzeichen.
 - Beispiel: 24b9da6552252987aa493b52f8696cd6d3b00373
- Git nutzt diese Hashwerte sehr intensiv, etwa als **Bezeichnungen für die Einträge** in seiner Datenbank (es werden keine Namen verwendet)!
- Angabe **vollständiger Prüfsumme nicht nötig**:
Solange mindestens die ersten vier Zeichen angegeben werden und der angegebene Teil der Prüfsumme eindeutig ist, reichen viel weniger Zeichen!
 - Oft: Die ersten 8-10 Zeichen reichen, um Objekt eindeutig zu kennzeichnen
 - Linux-Kernel: 11 Zeichen reichen!

Git: Mögliche Zustände einer Datei



Staged = „Markiert zur Aufnahme im nächsten Commit-Snapshot“

Verzeichnisse bei Git



Git Repositorium

- Spezielles Verzeichnis .git

 branches	17.09.2013 14:25	Dateiordner	
 hooks	17.09.2013 14:25	Dateiordner	
 logs	17.09.2013 14:25	Dateiordner	
 objects	17.09.2013 15:08	Dateiordner	
 refs	17.09.2013 14:25	Dateiordner	
 COMMIT_EDITMSG	17.09.2013 15:08	Datei	1 KB
 config	17.09.2013 15:08	Datei	1 KB
 FETCH_HEAD	17.09.2013 14:25	Datei	1 KB
 HEAD	18.09.2013 09:10	Datei	1 KB
 index	18.09.2013 09:10	Datei	9 KB
 ORIG_HEAD	17.09.2013 15:08	Datei	1 KB
 tortoisegit.data	18.09.2013 09:13	DATA-Datei	30 KB
 tortoisegit.index	18.09.2013 09:13	INDEX-Datei	1 KB

Wichtige Git-Befehle

`git [Befehl][Dateien]`, also `git init`, `git add myfile.txt`

- **init** – Erzeugt leeres Repositorium in Unterverzeichnis des aktuellen Verzeichnis. Aktuelles Verzeichnis wird Working Directory. Es wird noch nichts getrackt.
- **status** – Status der eigenen Dateien anzeigen
- **add** – fügt veränderte Dateien oder neue Dateien (auch eine Veränderung!) ins Staging für späteren Commit hinzu
 - Dateien werden so ins Staging übernommen, wie add sie vorfindet
 - Werden nach add weitere Veränderungen vorgenommen, fehlen diese beim auf das add folgenden commit, wenn nicht vorher ein erneutes add ausgeführt wird
- **reset** – Datei aus Staging wieder entfernen
- **commit -m "Message"** – Erzeugt neuen Snapshot im Repositorium, der aus den Dateien im Staging-Bereich besteht, mit Protokoll „Message“
- **rm** – Datei wieder aus Repositorium entfernen (aus Working Directory löschen reicht nicht!)

Beispiel – Git lokal benutzen

Mit folgenden Teilschritten:

- Repository anlegen
- Datei erzeugen
- Datei und spätere Änderungen committen
- Änderungen zurücknehmen
- Änderungshistorie ansehen

Beispiel

- Repository anlegen
- Datei erzeugen und committen
- Änderungen committen
- Änderungen zurücknehmen
- Änderungshistorie ansehen

```
Welcome to Git (version 1.8.3-preview20130601)

Run 'git help git' to display the help index.
Run 'git help <command>' to display help for specific commands.

rschimka@RSCHIMKA-PC ~
$ cd d:/tmp

rschimka@RSCHIMKA-PC /d/tmp
$ mkdir myfirstrepo

rschimka@RSCHIMKA-PC /d/tmp
$ cd myfirstrepo/

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo
$ echo "Meine erste Datei." > myfile.txt

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo
$ ls
myfile.txt

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo
$ git init
Initialized empty Git repository in d:/tmp/myfirstrepo/.git/

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ ls
myfile.txt

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ ls -al
total 5
drwxr-xr-x  1 rschimka Administ      0 Sep 18 08:14 .
drwxr-xr-x 12 rschimka Administ 4096 Sep 18 08:13 ..
drwxr-xr-x  1 rschimka Administ 4096 Sep 18 08:14 .git
-rw-r--r--  1 rschimka Administ   19 Sep 18 08:13 myfile.txt
```

Beispiel

- Repository anlegen
- Datei erzeugen
- Datei und spätere Änderungen committen
- Änderungen zurücknehmen
- Änderungshistorie ansehen

```
$ git commit -am "Initialer Commit"
# On branch master
#
# Initial commit
#
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#
#       myfile.txt
nothing added to commit but untracked files present (use "git add" to track)

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git add myfile.txt
warning: LF will be replaced by CRLF in myfile.txt.
The file will have its original line endings in your working directory.

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git commit -am "Initialer Commit"
[master (root-commit) a51b79a] Initialer Commit
warning: LF will be replaced by CRLF in myfile.txt.
The file will have its original line endings in your working directory.
1 file changed, 1 insertion(+)
create mode 100644 myfile.txt

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git status
# On branch master
nothing to commit, working directory clean
```

Beispiel

- Repository anlegen
- Datei erzeugen
- Datei und spätere Änderungen committen
- Änderungen zurücknehmen
- Änderungshistorie ansehen

```
rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ echo "Meine ersten Aenderungen" >> myfile.txt

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git status
# On branch master
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
#       modified:   myfile.txt
#
no changes added to commit (use "git add" and/or "git commit -a")

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ ls
myfile.txt

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ echo "Meine zweiten Aenderungen" >> myfile.txt

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git status
# On branch master
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
#       modified:   myfile.txt
#
no changes added to commit (use "git add" and/or "git commit -a")

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git diff
diff --git a/myfile.txt b/myfile.txt
index 70aa8d7..179c463 100644
--- a/myfile.txt
+++ b/myfile.txt
@@ -1,3 @@
  Meine erste Datei.
+Meine ersten Aenderungen
+Meine zweiten Aenderungen
warning: LF will be replaced by CRLF in myfile.txt.
The file will have its original line endings in your working directory.
```

Beispiel

- Repository anlegen
- Datei erzeugen
- Datei und spätere Änderungen committen
- Änderungen zurücknehmen
- Änderungshistorie ansehen

```
rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git commit -m "was wird hier committed?"
# On branch master
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
#       modified:   myfile.txt
#
no changes added to commit (use "git add" and/or "git commit -a")

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git add myfile.txt
warning: LF will be replaced by CRLF in myfile.txt.
The file will have its original line endings in your working directory.

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git status
warning: LF will be replaced by CRLF in myfile.txt.
The file will have its original line endings in your working directory.
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#       modified:   myfile.txt
#

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git commit -m "jetzt kann ich committen"
[master warning: LF will be replaced by CRLF in myfile.txt.
The file will have its original line endings in your working directory.
cc0c2f8] jetzt kann ich committen
warning: LF will be replaced by CRLF in myfile.txt.
The file will have its original line endings in your working directory.
1 file changed, 2 insertions(+)

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git status
# On branch master
nothing to commit, working directory clean
```

Beispiel

- Repository anlegen
- Datei erzeugen
- Datei und spätere Änderungen committen
- Änderungen zurücknehmen
- Änderungshistorie ansehen

```
rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ echo "weitere Aenderungen" >> myfile.txt

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git add myfile.txt
warning: LF will be replaced by CRLF in myfile.txt.
The file will have its original line endings in your working directory.

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git reset HEAD myfile.txt
warning: LF will be replaced by CRLF in myfile.txt.
The file will have its original line endings in your working directory.
Unstaged changes after reset:
M      myfile.txt
```

```
rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git status
# On branch master
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
#       modified:   myfile.txt
#
no changes added to commit (use "git add" and/or "git commit -a")

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git checkout myfile.txt
```

Beispiel

- Repository anlegen
- Datei erzeugen
- Datei und spätere Änderungen committen
- Änderungen zurücknehmen
- Änderungshistorie ansehen

```
rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$ git log
commit cc0c2f844b33a9ac3362136626938dab051b5e8e
Author: rds <rds@schimkat.org>
Date:   Wed Sep 18 10:26:03 2013 +0200

    jetzt kann ich committen

commit a51b79a50c31d9e12fc1409f6bb1e75177cf464f
Author: rds <rds@schimkat.org>
Date:   Wed Sep 18 10:15:23 2013 +0200

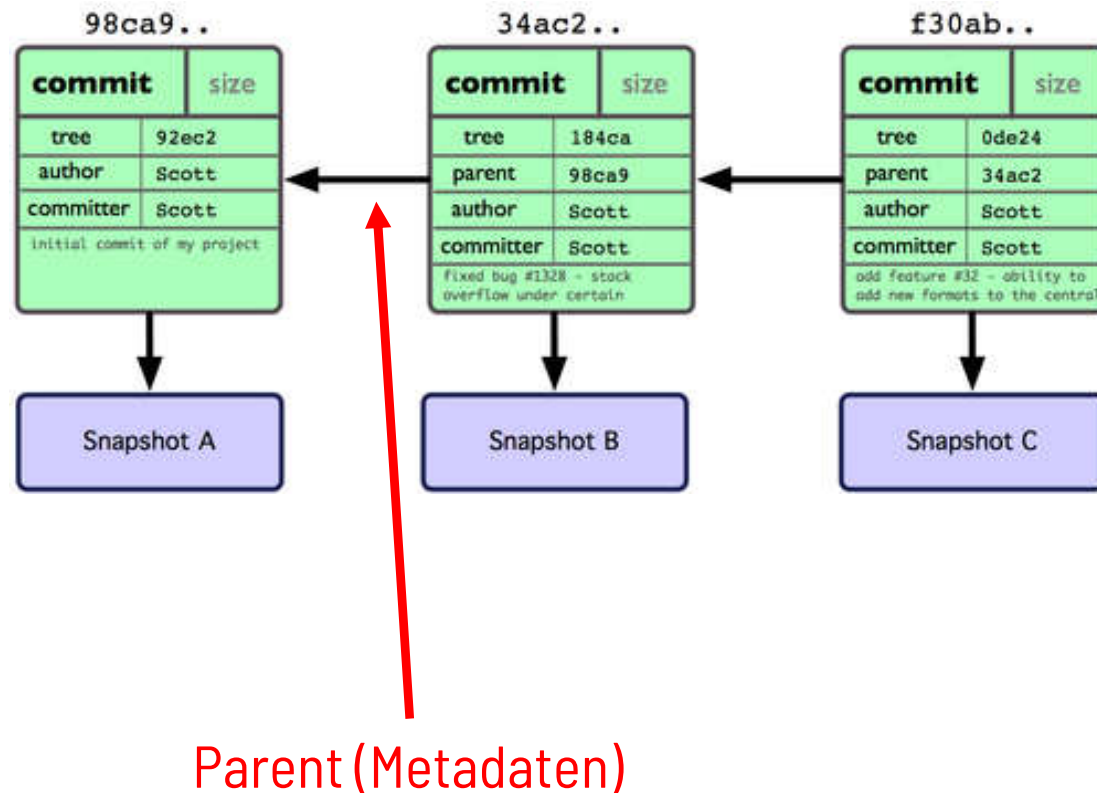
    Initialer Commit

rschimka@RSCHIMKA-PC /d/tmp/myfirstrepo (master)
$
```

Git, *Repositories* und Branches



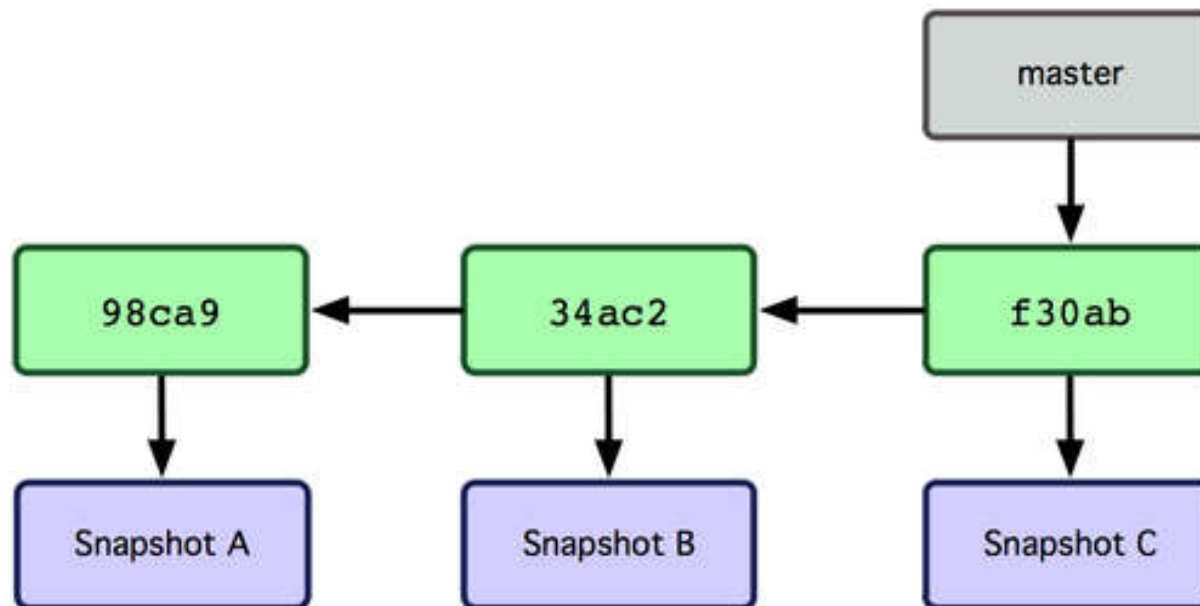
Ein Git-Repository läßt sich als Menge von Commits sehen:



Git, Repositories und *Branches*



- Ein Branch (Verzweigung) bei Git ist ein Zeiger auf einen Commit

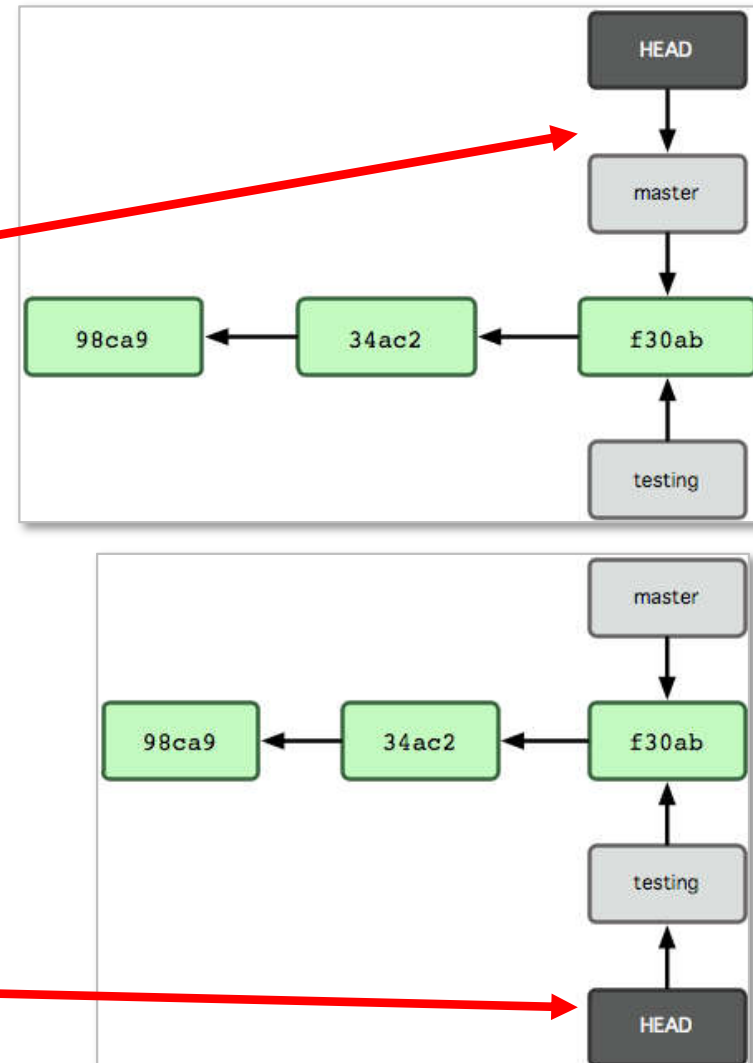


- Der Standard-Branch heißt *master*

Git, Repositories und *Branches*



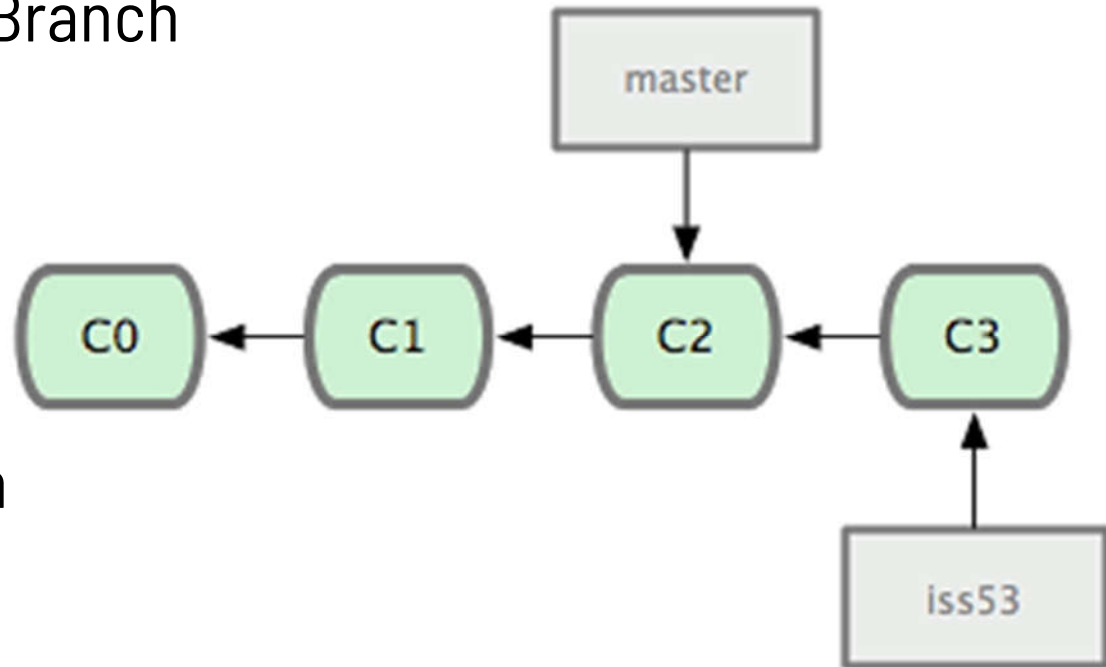
- HEAD zeigt stets auf den aktuell bearbeiteten Stand im Arbeitsverzeichnis
- Standardmässig zeigt HEAD auf `master`
- Mehrere Branches können auf denselben Snapshot zeigen
- Mit `git checkout` kann HEAD „umgelenkt“ werden, hier mit `git checkout testing` auf `testing`



Git, Repositories und *Branches*

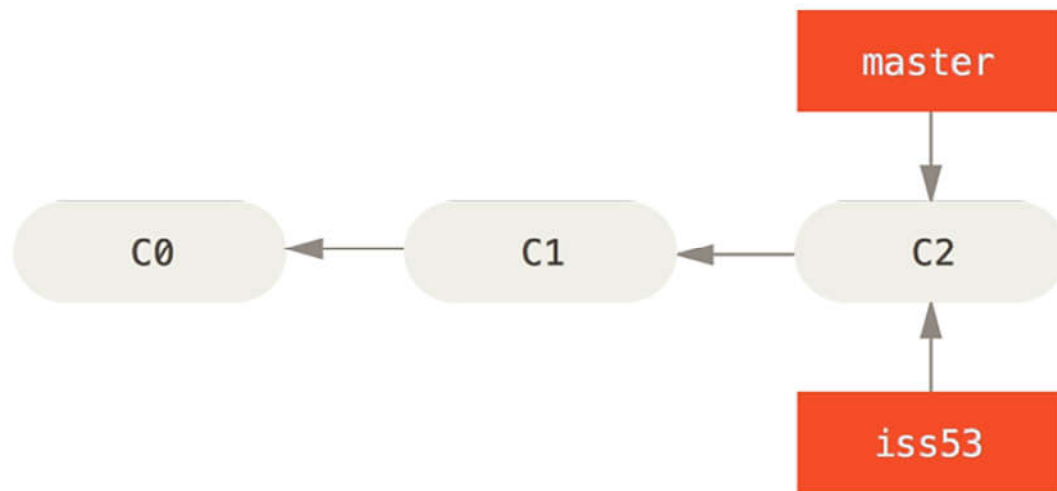


Jeder weitere Commit führt HEAD fort – auf dem aktuellen Branch

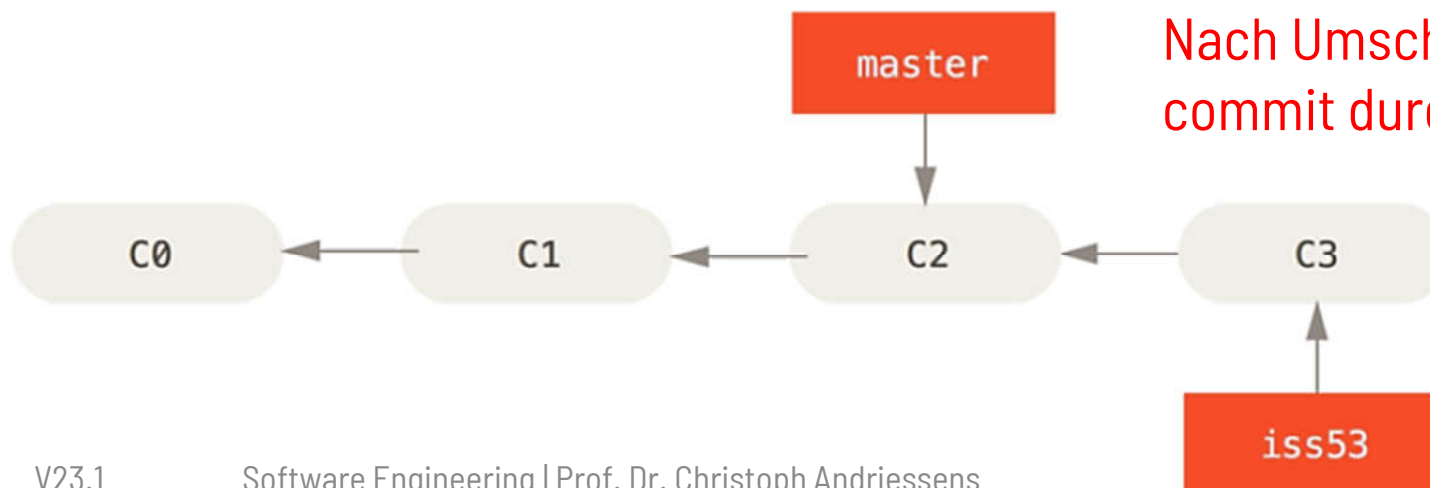


Hier können Zweige dann auseinanderlaufen

Git, Repositories und *Branches: Anlegen*

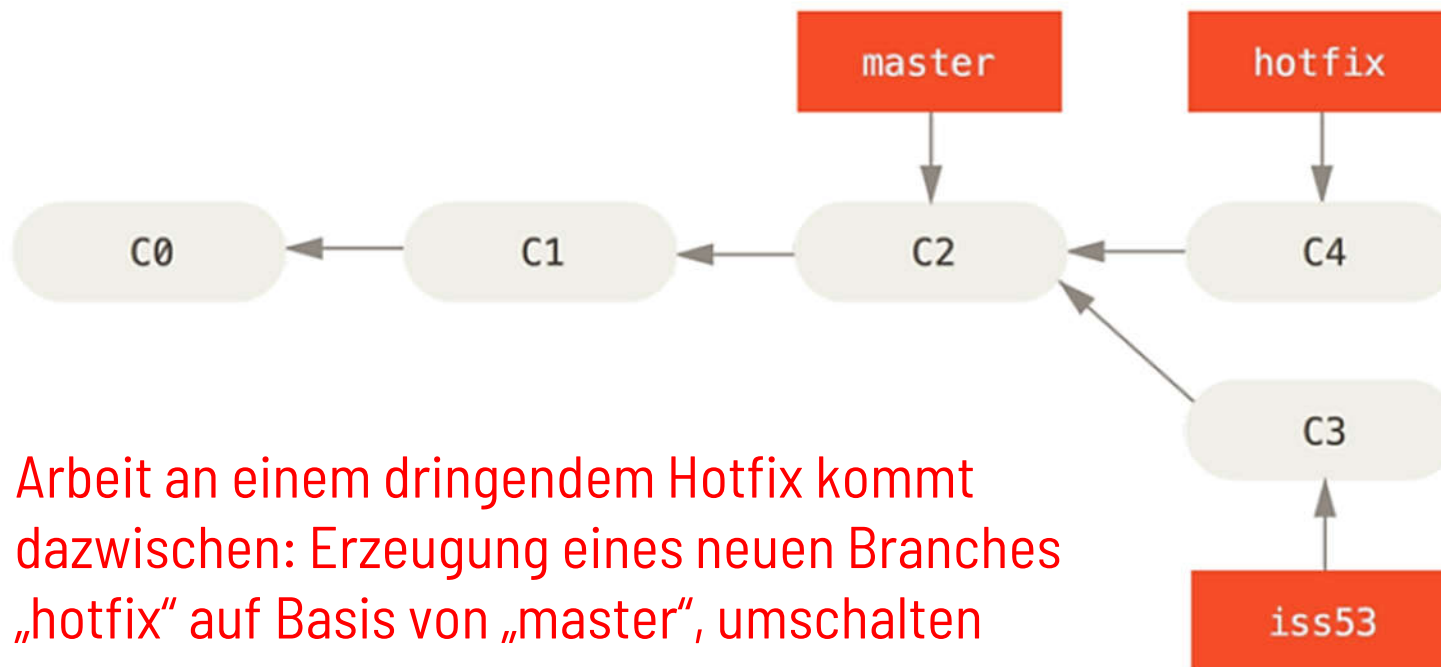


Neuer Branch „iss53“ angelegt,
um an „Issue 53“ zu arbeiten:
`git branch iss53`
`git checkout iss53`



Nach Umschalten auf „iss53“ ein
commit durchgeführt

Git, Repositories und *Branches: Anlegen*



Arbeit an einem dringendem Hotfix kommt
dazwischen: Erzeugung eines neuen Branches
„hotfix“ auf Basis von „master“, umschalten
darauf und Durchführung eines Commits

```
git checkout master
```

```
git branch hotfix
```

```
git checkout hotfix
```

...

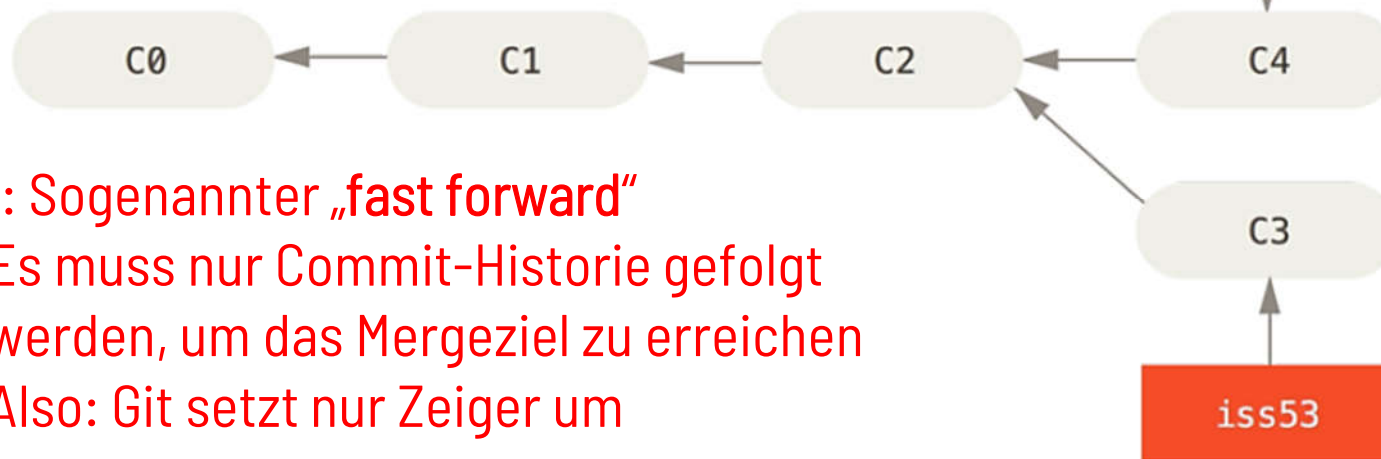
Git, Repositories und *Branches*: Mergen in Git

„Mergen“ heißt:

Änderungen zwischen unterschiedlichen Versionsständen unterschiedlicher Branches werden zusammengeführt

Beispiel – Mergen in Git 1/3

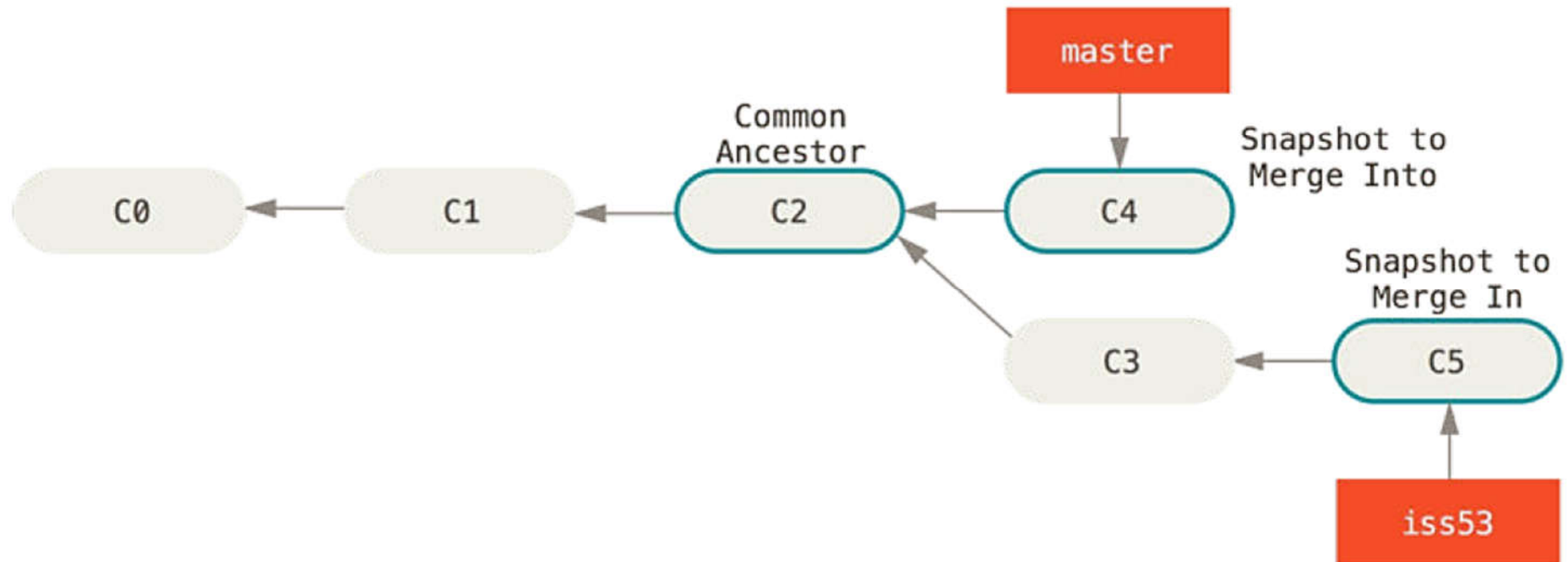
- Wechsle auf den Branch, der Merge-Ergebnis enthalten soll
 - `git checkout master`
- Merge Branch hotfix
 - `git merge hotfix`



Hier: Sogenannter „fast forward“

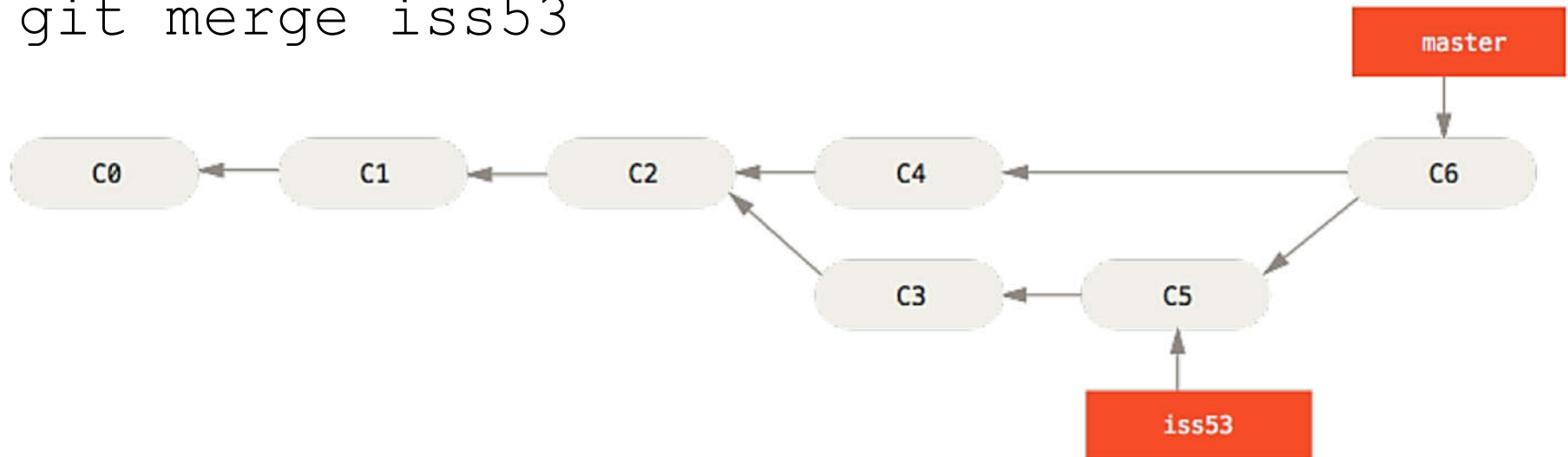
- Es muss nur Commit-Historie gefolgt werden, um das Mergeziel zu erreichen
- Also: Git setzt nur Zeiger um

Beispiel – Mergen in Git 2/3



Beispiel – Mergen in Git 3/3

- Wechsle auf den zu mergenden Branch
 - `git checkout master`
- Merge Branch iss53
 - `git merge iss53`



Merge-Konflikte

```
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")

Unmerged paths:
  (use "git add <file>..." to mark resolution)

    both modified:    index.html

no changes added to commit (use "git add" and/or "git commit -a")
```

Merge-Konflikte

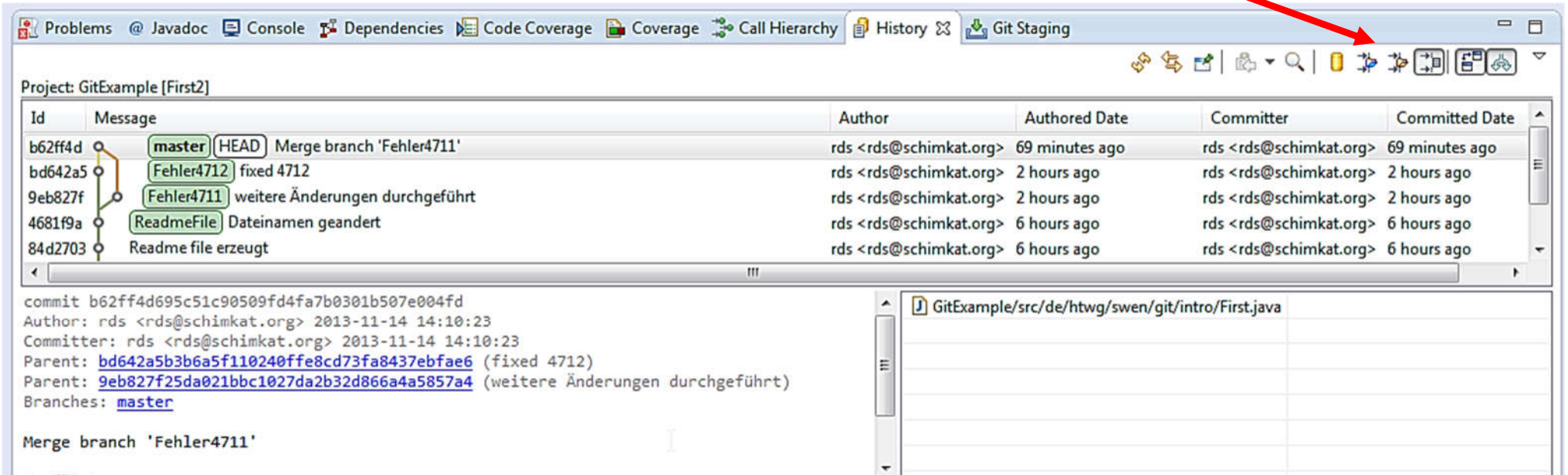
So sieht ein Merge-Konflikt dann in den Dateien aus:

```
<<<<<<< HEAD:index.html
<div id="footer">contact : email.support@github.com</div>
=====
<div id="footer">
  please contact us at support@github.com
</div>
>>>>>>> iss53:index.html
```

Oben der Anteil aus HEAD, unten der aus iss53.

Dieser Abschnitt muss von Hand ersetzt werden durch sinnvolle Lösung, dann `git add` und `git commit`.

Vergleichen in Eclipse mit Git



Project: GitExample [First2]

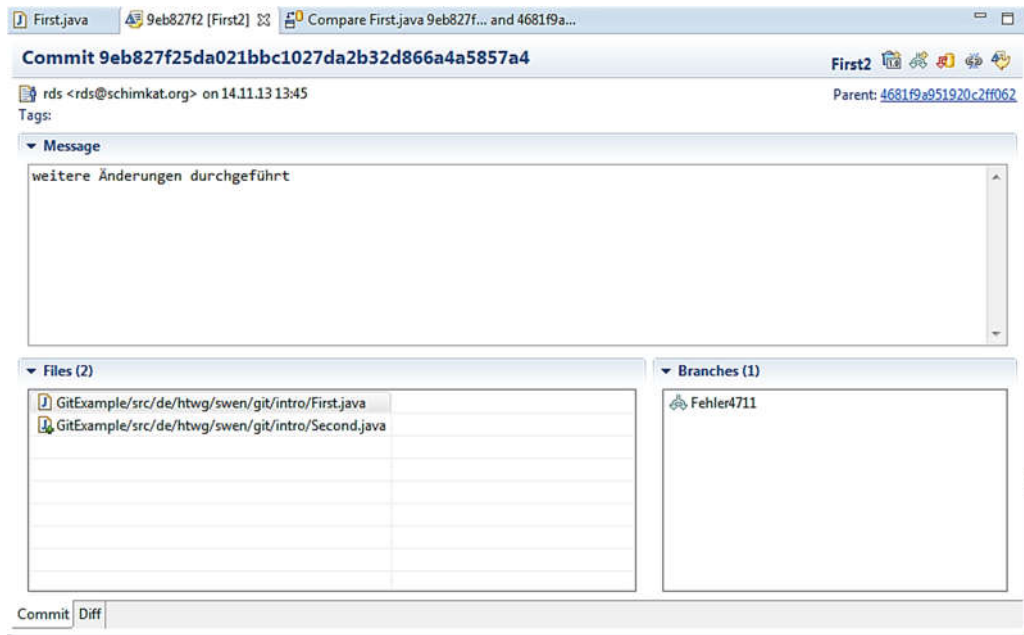
Id	Message	Author	Authored Date	Committer	Committed Date
b62ff4d	master HEAD Merge branch 'Fehler4711'	rds <rds@schimkat.org>	69 minutes ago	rds <rds@schimkat.org>	69 minutes ago
bd642a5	Fehler4712 fixed 4712	rds <rds@schimkat.org>	2 hours ago	rds <rds@schimkat.org>	2 hours ago
9eb827f	Fehler4711 weitere Änderungen durchgeführt	rds <rds@schimkat.org>	2 hours ago	rds <rds@schimkat.org>	2 hours ago
4681f9a	ReadmeFile Dateinamen geändert	rds <rds@schimkat.org>	6 hours ago	rds <rds@schimkat.org>	6 hours ago
84d2703	Readme file erzeugt	rds <rds@schimkat.org>	6 hours ago	rds <rds@schimkat.org>	6 hours ago

commit b62ff4d695c51c90509fd4fa7b0301b507e004fd
Author: rds <rds@schimkat.org> 2013-11-14 14:10:23
Committer: rds <rds@schimkat.org> 2013-11-14 14:10:23
Parent: [bd642a5b3b6a5f110240ffe8cd73fa8437ebfae6](#) (fixed 4712)
Parent: [9eb827f25da021bbc1027da2b32d866a4a5857a4](#) (weitere Änderungen durchgeführt)
Branches: [master](#)

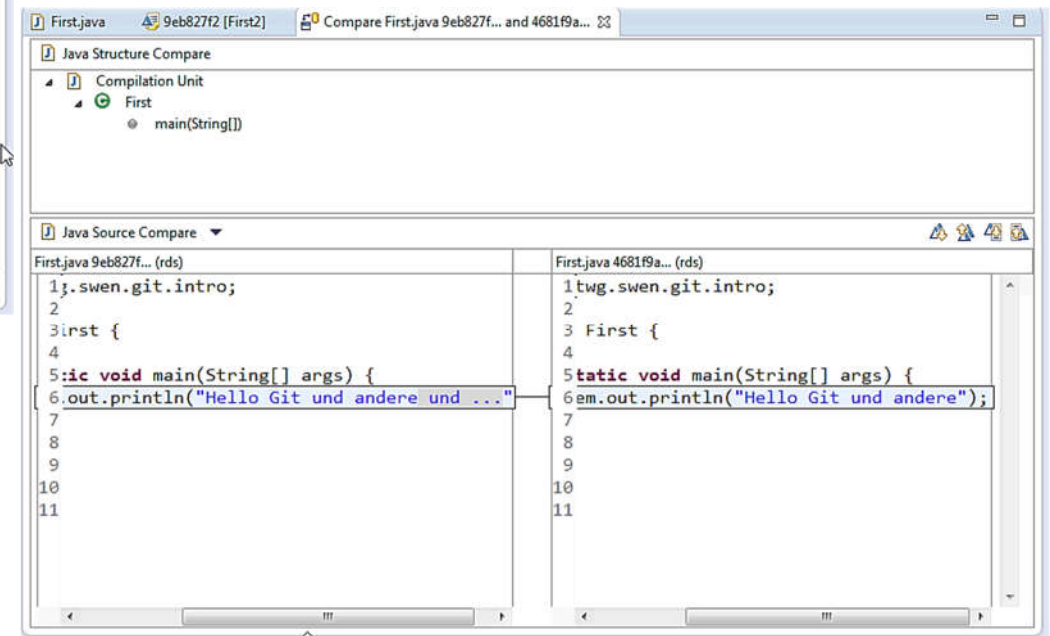
Merge branch 'Fehler4711'

GitExample/src/de/htwg/swen/git/intro/First.java

Vergleichen in Eclipse mit Git



Details der Änderungen im Vergleichseditor betrachten



Git „remote“ benutzen

Ein Repository auf einem Server einrichten, damit mehrere Personen darauf zugreifen können

- Einrichten eines Master-Repositories
- jeder Entwickler verwendet zusätzlich eigenes lokales Repository

➤ Kollaboratives, verteiltes Arbeiten

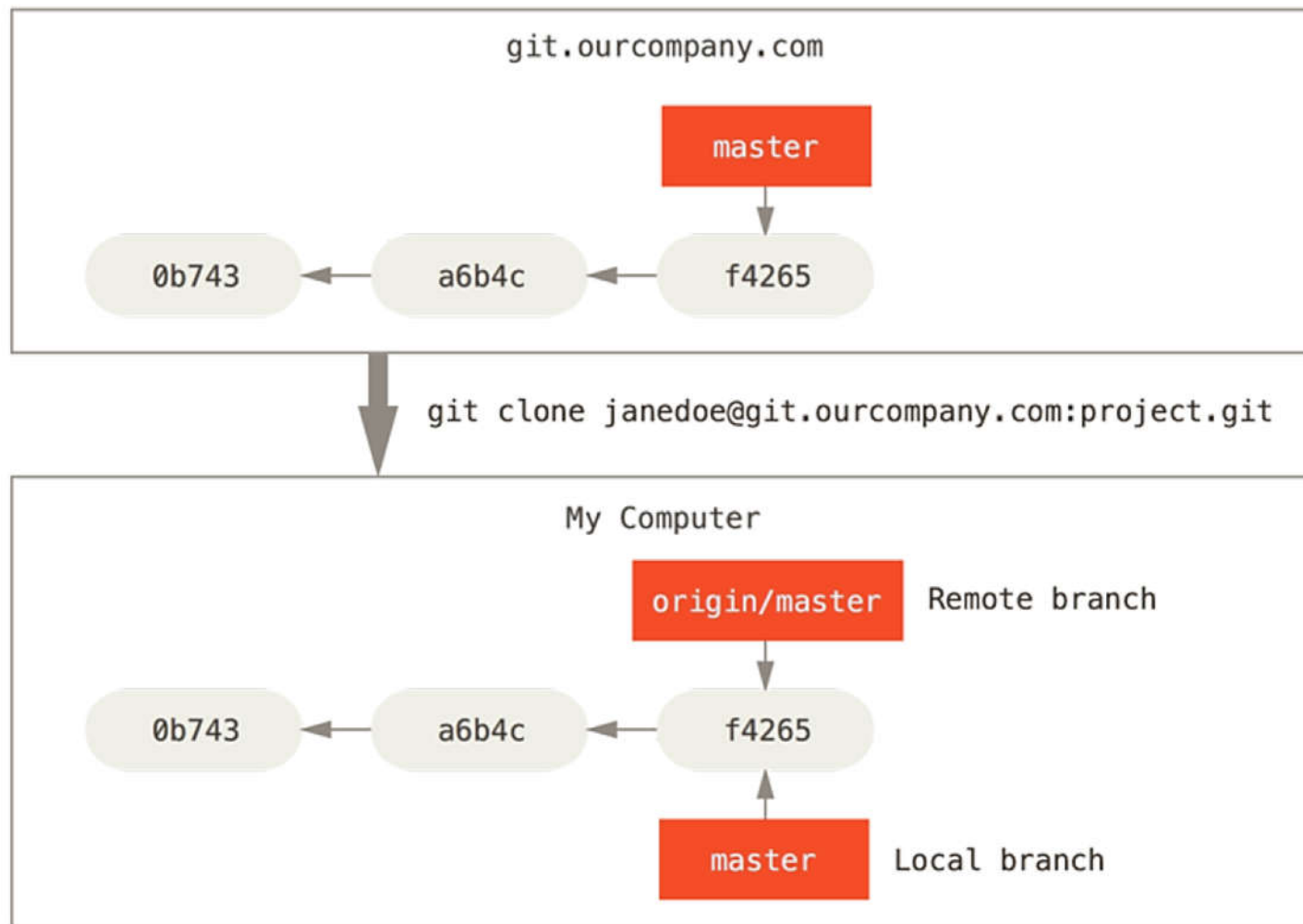
Git „remote“ benutzen

Befehle in Zusammenarbeit lokales / entferntes Repository:

- **clone:** Klont das entfernte Repository in ein lokales Repository, z.B.
 - `git clone ↵`
`https://git.hs-weingarten.de/scm/git/git-html-demo`
- **fetch:** holt alle Daten aus dem entfernten Repository, die noch nicht im lokalen sind
- **pull:** Wie *fetch*, versucht dazu anschließend automatisch ein *merge*
- **push:** Lokales Repository (oder einzelnen Branch daraus) in entferntes Repository kopieren
 - Funktioniert nur, wenn im entfernten Rep. seit letzten clone, fetch oder pull keine Veränderungen aufgetreten sind!
 - Sonst erst fetch & merge notwendig
- **remote:** Management von entfernten Repositories, z.B.
 - Git-Repository unter angegebener URL als entf. Repository „origin“ angeben:
`git remote add origin ↵`
`https://git.hs-weingarten.de/scm/git/git-html-demo`

Git „remote“ benutzen

Beispiel: Klonen und Branches (1/3)



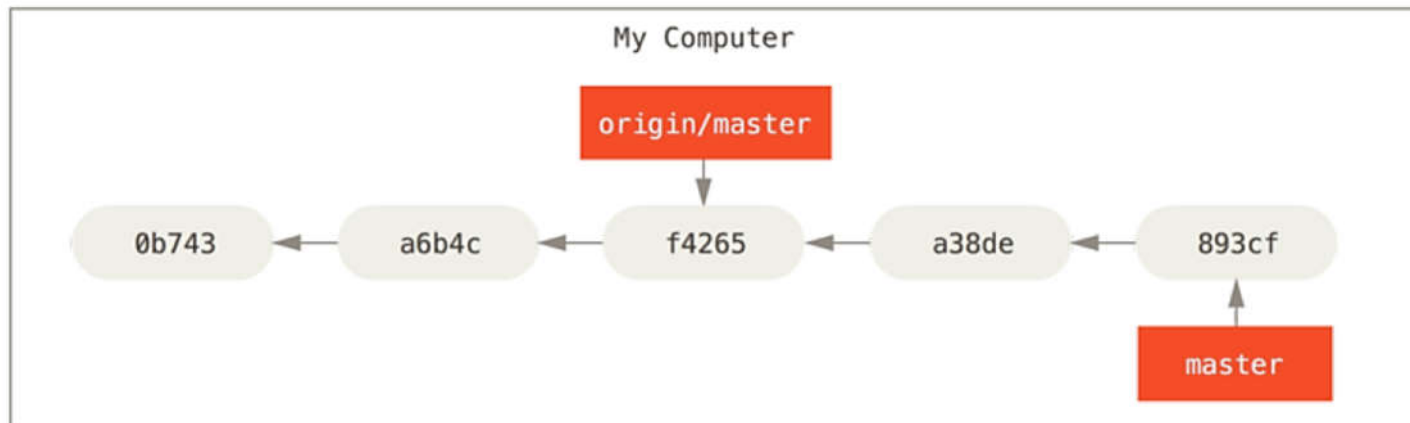
Git nennt
entferntes
Repository
automatisch
„origin“,
holt alle Daten,
setzt Zeiger auf
entfernten
Master-Branch
mit Namen
„origin/master“
und erzeugt
lokalen Master-
Branch „master“

Git „remote“ benutzen

Beispiel: Klonen und Branches (2/3)



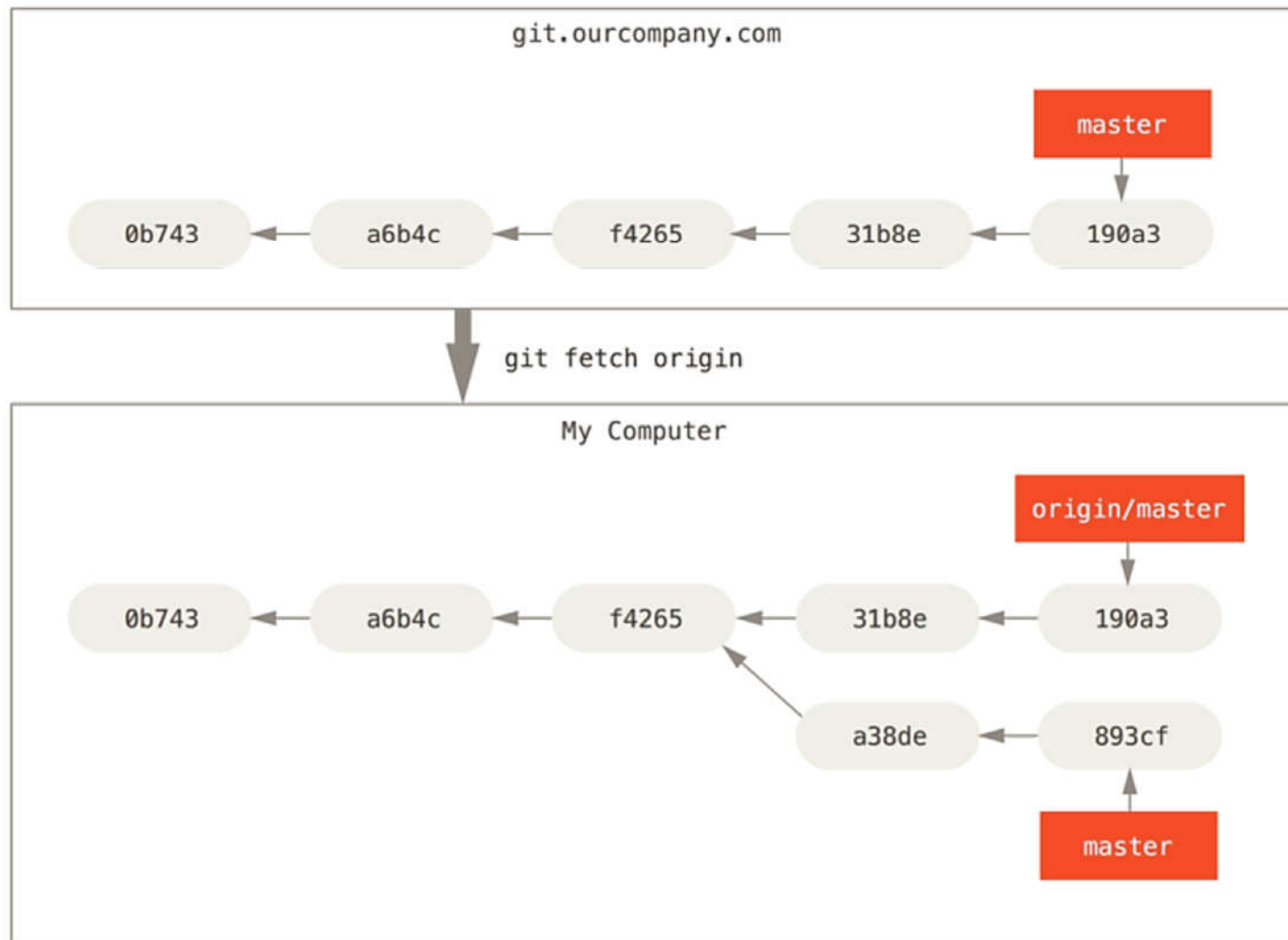
Andere Entwickler arbeiten und pushen ihre Ergebnisse



Im lokalen Repository kommen ebenfalls commits hinzu,

Git „remote“ benutzen

Beispiel: Klonen und Branches (3/3)



fetch
aktualisiert
das lokale
Repository

Beispiel: Git-Master-Repository eines Teams aus dem Software Engineering-Praktikum

master

Anonymisierter Ausschnitt

Date	Message
2017-01-07 03-25-01	Story #1565 implementier
2017-01-04 23-04-47	Story #1563 implementier
2016-12-21 04-11-35	Bestellung/Bestellpositionstornierung
2016-12-20 14-37-11	cancelOrderPosition angelegt (Service)
2016-12-20 14-09-03	Merge branch 'M██████'
2016-12-20 14-07-50	Anzahl Stornierungen von Produkt und Kunde auf Serviceschicht
2016-12-20 13-57-31	Merge branch 'T██████'
2016-12-20 13-55-48	State-Pattern hinzugefügt + Tests
2016-12-20 12-19-53	State-Pattern
2016-12-20 12-14-02	OrderCancelation OrderPositionCancelation erstellt

Beispiel: Git-Master-Repository eines Teams aus dem Software Engineering-Praktikum

Anonymisierter Ausschnitt

T [REDACTED]

Date	Message
2016-12-20 12-19-53	State-Pattern
2016-12-19 15-23-01	addOrder() mit Test implementiert
2016-12-19 01-39-51	Bugfixes
2016-12-18 03-19-51	GetOrderById() auf Datenbankebene implementiert und erfolgreich getestet
2016-12-18 00-02-40	Zeugs
2016-12-16 14-03-38	Unused Imports entfernt, Rechtschreibfehler korrigiert und Orders Tabelle hinzugefügt
2016-12-16 13-42-01	Merge branch 'M [REDACTED]'
2016-12-16 13-38-30	Unused Import entfernt und Orders Tabelle hinzugefügt
2016-12-15 16-04-56	getCustomerById + ReadInterface DBUnit Connect / XML Files Aenderung
2016-12-14 11-46-20	DBCustomerRead und Write Interface angelegt und Klasen angepasst

Git online als Spiel üben

<https://learngitbranching.js.org/>

The screenshot shows the 'Learn Git Branching' interface. At the top, there's a header 'Learn Git Branching' and a 'Fork me on GitHub' button. Below this is a 'Git Demonstration' window. On the left side of the window, there's a text area with German instructions: 'Hier haben wir zwei Branches; jeder besteht jeweils aus einem eigenen Commit. Das bedeutet, dass keiner der beiden Branches alle Inhalte des gesamten Repositorys kennt. Das werden wir mit einem Merge ändern. Wir werden den Branch `bugFix` in `master` integrieren'. Below the text is a green button with the command `git merge bugFix`. On the right side of the window, there's a diagram of the Git commit history. It shows a vertical line of commits: `c0` (purple), `c1` (purple), `c2` (green), and `c3` (pink). Arrows indicate the flow: `c1` points to `c0`, `c2` points to `c1`, and `c3` points to `c1`. A green box labeled `bugFix` is positioned between `c2` and `c1`, and a pink box labeled `master*` is positioned between `c3` and `c1`. At the bottom of the window, there are two large circular buttons: a red one with a left arrow and a green one with a right arrow. The bottom right corner of the screen shows a system tray with icons for help, home, and social media.

Zusammenfassung

- Versionsverwaltungssysteme sind ein notwendiges Werkzeug für die Entwicklung von Software in Teams
- Git ist ein mächtiges Werkzeug zur effizienten und effektiven Versionsverwaltung

